

The essential software requirement

"Hello, Phil? This is Maria in Human Resources. We're having a problem with the personnel system you programmed for us. An employee just changed her name to Sparkle Starlight, and we can't get the system to accept the name change. Can you help?"

"She married some guy named Starlight?"

"No, she didn't get married, just changed her name," Maria replied. "That's the problem. It looks like we can change a name only if someone's marital status changes."

"Well, yeah, I never thought someone might just change her name. I don't remember you telling me about this possibility when we talked about the system," Phil said.

"I assumed you knew that people could legally change their name anytime they like," responded Maria. "We have to straighten this out by Friday or Sparkle won't be able to cash her paycheck. Can you fix the bug by then?"

"It's not a bug!" Phil retorted. "I never knew you needed this capability. I'm busy on the new performance evaluation system. I can probably fix it by the end of the month, but not by Friday. Sorry about that. Next time, tell me these things earlier and please write them down."

"What am I supposed to tell Sparkle?" demanded Maria. "She'll be upset if she can't cash her check."

"Hey, Maria, it's not my fault," Phil protested. "If you'd told me in the first place that you had to be able to change someone's name at any time, this wouldn't have happened. You can't blame me for not reading your mind."

Angry and resigned, Maria snapped, "Yeah, well, this is the kind of thing that makes me hate computers. Call me as soon as you get it fixed, will you?"

If you've ever been on the customer side of a conversation like this, you know how frustrating it is when a software system doesn't let you perform an essential task. You hate to be at the mercy of a developer who *might* get to your critical change request eventually. On the other hand, developers are frustrated to learn about functionality that a user expected only after they've implemented the system. It's also annoying for a developer to have his current project interrupted by a request to modify a system that does precisely what he was told it should do in the first place.

Many problems in the software world arise from shortcomings in the ways that people learn about, document, agree upon and modify the product's requirements. As with Phil and Maria, common problem areas are informal information gathering, implied functionality, miscommunicated assumptions, poorly specified requirements, and a casual change process. Various studies suggest that errors introduced during requirements activities account for 40 to 50 percent of all defects found in a software product (Davis 2005). Inadequate user input and shortcomings in specifying and managing customer requirements are major contributors to unsuccessful projects. Despite this evidence, many organizations still practice ineffective requirements methods.

Nowhere more than in the requirements do the interests of all the stakeholders in a project intersect. (See Chapter 2, "Requirements from the customer's perspective," for more about stakeholders.) These stakeholders include customers, users, business analysts, developers, and many others. Handled well, this intersection can lead to delighted customers and fulfilled developers. Handled poorly, it's the source of misunderstanding and friction that undermine the product's quality and business value. Because requirements are the foundation for both the software development and the project management activities, all stakeholders should commit to applying requirements practices that are known to yield superior-quality products.

But developing and managing requirements is hard! There are no simple shortcuts or magic solutions. On the plus side, so many organizations struggle with the same problems that you can look for techniques in common that apply to many different situations. This book describes dozens of such practices. The practices are presented as though you were building a brand-new system. However, most of them also apply to enhancement, replacement, and reengineering projects (see Chapter 21, "Enhancement and replacement projects") and to projects that incorporate commercial off-the-shelf (COTS) packaged solutions (see Chapter 22, "Packaged solution projects"). Project teams that build products incrementally by following an agile development process also need to understand the requirements that go into each increment (see Chapter 20, "Agile projects").

This chapter will help you to:

- Understand some key terms used in the software requirements domain.
- Distinguish *product* requirements from *project* requirements.
- Distinguish requirements *development* from requirements *management*.
- Be alert to several requirements-related problems that can arise.



Important We use the terms "system," "product," "application," and "solution" interchangeably in this book to refer to any kind of software or software-containing item that you build, whether for internal corporate use, for commercial sale, or on a contract basis.

Taking your requirements pulse

For a quick check of the current requirements practices in your organization, consider how many of the following conditions apply to your most recent project. If more than three or four of these items describe your experience, this book is for you:

- The project's business objectives, vision, and scope were never clearly defined.
- Customers were too busy to spend time working with analysts or developers on the requirements.
- Your team could not interact directly with representative users to understand their needs.
- Customers claimed that all requirements were critical, so they didn't prioritize them.
- Developers encountered ambiguities and missing information when coding, so they had to guess.
- Communications between developers and stakeholders focused on user interface displays or features, not on what users needed to accomplish with the software.
- Your customers never approved the requirements.
- Your customers approved the requirements for a release or iteration and then changed them continually.
- The project scope increased as requirements changes were accepted, but the schedule slipped because no additional resources were provided and no functionality was removed.
- Requested requirements changes got lost; no one knew the status of a particular change request.
- Customers requested certain functionality and developers built it, but no one ever uses it.
- At the end of the project, the specification was satisfied but the customer or the business objectives were not.

Software requirements defined

When a group of people begin discussing requirements, they often start with a terminology problem. Different observers might describe a single statement as being a user requirement, software requirement, business requirement, functional requirement, system requirement, product requirement, project requirement, user story, feature, or constraint. The names they use for various requirements deliverables also vary. A customer's definition of requirements might sound like a high-level product concept to the developer. The developer's notion of requirements might sound like a detailed user interface design to the user. This diversity of understanding leads to confusion and frustration.

Some interpretations of “requirement”

Many decades after the invention of computer programming, software practitioners still have raging debates about exactly what a “requirement” is. Rather than prolong those debates, in this book we simply present some definitions that we have found useful.

Consultant Brian Lawrence suggests that a *requirement* is “anything that drives design choices” (Lawrence 1997). This is not a bad colloquial definition, because many kinds of information fit in this category. And, after all, the whole point of developing requirements is to make appropriate design choices that will meet the customer’s needs in the end. Another definition is that a requirement is a property that a product must have to provide value to a stakeholder. Also not bad, but not very precise. Our favorite definition, though, comes from Ian Sommerville and Pete Sawyer (1997):

Requirements are a specification of what should be implemented. They are descriptions of how the system should behave, or of a system property or attribute. They may be a constraint on the development process of the system.

This definition acknowledges the diverse types of information that collectively are referred to as “the requirements.” Requirements encompass both the user’s view of the external system behavior and the developer’s view of some internal characteristics. They include both the behavior of the system under specific conditions and those properties that make the system suitable—and maybe even enjoyable—for use by its intended operators.

Trap Don’t assume that all your project stakeholders share a common notion of what requirements are. Establish definitions up front so that you’re all talking about the same things.

The pure dictionary “requirement”

Software people do not use “requirement” in the same sense as a dictionary definition of the word: something demanded or obligatory, a need or necessity. People sometimes question whether they even need to prioritize requirements, because maybe a low-priority requirement won’t ever be implemented. If it isn’t truly needed, then it isn’t a requirement, they claim. Perhaps, but then what would you call that piece of information? If you defer a requirement from today’s project to an unspecified future release, is it still considered a requirement? Sure it is.

Software requirements include a time dimension. They could be present tense, describing the current system’s capabilities. Or they could be for the near-term (high priority), mid-term (medium priority), or hypothetical (low priority) future. They could even be past tense, referring to needs that were once specified and then discarded. Don’t waste time debating whether or not something is a requirement, even if you know you might never implement it for some good business reason. It is.

Levels and types of requirements

Because there are so many different types of requirements information, we need a consistent set of adjectives to modify the overloaded term “requirement.” This section presents definitions we will use for some terms commonly encountered in the requirements domain (see Table 1-1).

TABLE 1-1 Some types of requirements information

Term	Definition
Business requirement	A high-level business objective of the organization that builds a product or of a customer who procures it.
Business rule	A policy, guideline, standard, or regulation that defines or constrains some aspect of the business. Not a software requirement in itself, but the origin of several types of software requirements.
Constraint	A restriction that is imposed on the choices available to the developer for the design and construction of a product.
External interface requirement	A description of a connection between a software system and a user, another software system, or a hardware device.
Feature	One or more logically related system capabilities that provide value to a user and are described by a set of functional requirements.
Functional requirement	A description of a behavior that a system will exhibit under specific conditions.
Nonfunctional requirement	A description of a property or characteristic that a system must exhibit or a constraint that it must respect.
Quality attribute	A kind of nonfunctional requirement that describes a service or performance characteristic of a product.
System requirement	A top-level requirement for a product that contains multiple subsystems, which could be all software or software and hardware.
User requirement	A goal or task that specific classes of users must be able to perform with a system, or a desired product attribute.

Software requirements include three distinct levels: business requirements, user requirements, and functional requirements. In addition, every system has an assortment of nonfunctional requirements. The model in Figure 1-1 illustrates a way to think about these diverse types of requirements. As statistician George E. P. Box famously said, “Essentially, all models are wrong, but some are useful” (Box and Draper 1987). That’s certainly true of Figure 1-1. This model is not all-inclusive, but it does provide a helpful scheme for organizing the requirements knowledge you’ll encounter.

The ovals in Figure 1-1 represent types of requirements information, and the rectangles indicate documents in which to store that information. The solid arrows indicate that a certain type of information typically is stored in the indicated document. (Business rules and system requirements are stored separately from software requirements, such as in a business rules catalog or a system requirements specification, respectively.) The dotted arrows indicate that one type of information is the origin of or influences another type of requirement. Data requirements are not shown explicitly in this diagram. Functions manipulate data, so data requirements can appear throughout the three levels. Chapter 7, “Requirements elicitation,” contains many examples of these different types of requirements information.

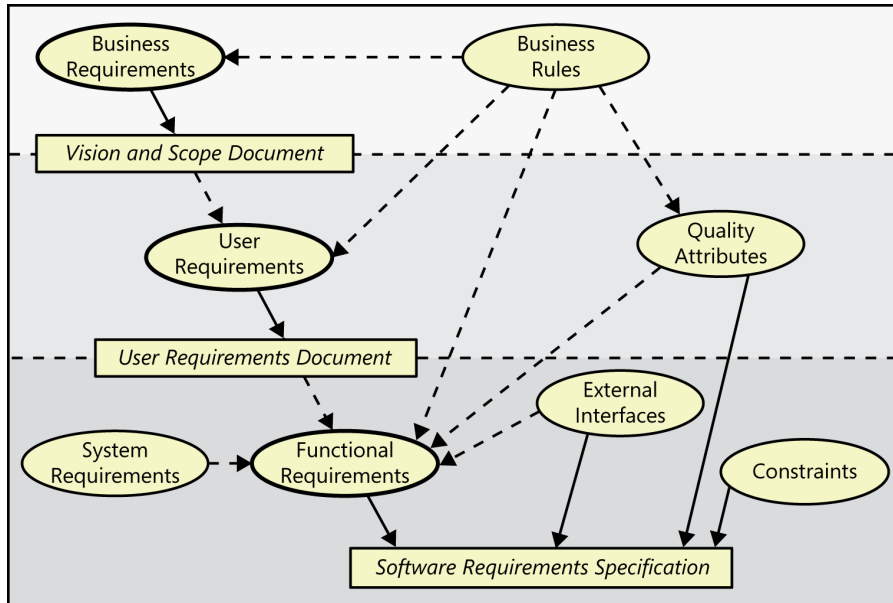


FIGURE 1-1 Relationships among several types of requirements information. Solid arrows mean “are stored in”; dotted arrows mean “are the origin of” or “influence.”



Important Although we will refer to requirements “documents” throughout this book, as in Figure 1-1, those do not have to be traditional paper or electronic documents. Instead, think of them simply as containers in which to store requirements knowledge. Such a container could indeed be a traditional document, or it could be a spreadsheet, a set of diagrams, a database, a requirements management tool, or some combination of these. For convenience, we will use the term “document” to refer to any such container. We will provide templates that identify the types of information to consider storing in each such grouping, regardless of what form you store it in. What you call each deliverable is less important than having your organization agree on their names, what kinds of information go into each, and how that information is organized.

Business requirements describe *why* the organization is implementing the system—the business benefits the organization hopes to achieve. The focus is on the business objectives of the organization or the customer who requests the system. Suppose an airline wants to reduce airport counter staff costs by 25 percent. This goal might lead to the idea of building a kiosk that passengers can use to check in for their flights at the airport. Business requirements typically come from the funding sponsor for a project, the acquiring customer, the manager of the actual users, the marketing department, or a product visionary. We like to record the business requirements in a *vision and scope document*. Other strategic guiding documents sometimes used for this purpose include a project charter, business case, and market (or marketing) requirements document. Specifying business requirements is the subject of Chapter 5, “Establishing the business requirements.” For the purposes of this book, we are assuming that the business need or market opportunity has already been identified.

User requirements describe goals or tasks the users must be able to perform with the product that will provide value to someone. The domain of user requirements also includes descriptions of product attributes or characteristics that are important to user satisfaction. Ways to represent user requirements include use cases (Kulak and Guiney 2004), user stories (Cohn 2004), and event-response tables. Ideally, actual user representatives will provide this information. User requirements describe *what* the user will be able to do with the system. An example of a use case is “Check in for a flight” using an airline’s website or a kiosk at the airport. Written as a user story, the same user requirement might read: “As a passenger, I want to check in for a flight so I can board my airplane.” It’s important to remember that most projects have multiple user classes, as well as other stakeholders whose needs also must be elicited. Chapter 8, “Understanding user requirements,” addresses this level of the model. Some people use the broader term “stakeholder requirements,” to acknowledge the reality that various stakeholders other than direct users will provide requirements. That is certainly true, but we focus the attention at this level on understanding what actual users need to achieve with the help of the product.

Functional requirements specify the behaviors the product will exhibit under specific conditions. They describe *what* the developers must implement to enable users to accomplish their tasks (user requirements), thereby satisfying the business requirements. This alignment among the three levels of requirements is essential for project success. Functional requirements often are written in the form of the traditional “shall” statements: “The Passenger shall be able to print boarding passes for all flight segments for which he has checked in” or “If the Passenger’s profile does not indicate a seating preference, the reservation system shall assign a seat.”

The business analyst (BA)¹ documents functional requirements in a *software requirements specification* (SRS), which describes as fully as necessary the expected behavior of the software system. The SRS is used in development, testing, quality assurance, project management, and related project functions. People call this deliverable by many different names, including business requirements document, functional spec, requirements document, and others. An SRS could be a report generated from information stored in a requirements management tool. Because it is an industry-standard term, we will use “SRS” consistently throughout this book (ISO/IEC/IEEE 2011). See Chapter 10, “Documenting the requirements,” for more information about the SRS.

System requirements describe the requirements for a product that is composed of multiple components or subsystems (ISO/IEC/IEEE 2011). A “system” in this sense is not just any information system. A system can be all software or it can include both software and hardware subsystems. People and processes are part of a system, too, so certain system functions might be allocated to human beings. Some people use the term “system requirements” to mean the detailed requirements for a software system, but that’s not how we use the term in this book.

A good example of a “system” is the cashier’s workstation in a supermarket. There’s a bar code scanner integrated with a scale, as well as a hand-held bar code scanner. The cashier has a keyboard, a display, and a cash drawer. You’ll see a card reader and PIN pad for your loyalty card and credit or debit card, and perhaps a change dispenser. You might see up to three printers for your purchase

¹ “Business analyst” refers to the project role that has primary responsibility for leading requirements-related activities on a project. The BA role also goes by many other names. See Chapter 4, “The business analyst,” for more about the business analyst role.

receipt, credit card receipt, and coupons you don't care about. These hardware devices are all interacting under software control. The requirements for the system or product as a whole, then, lead the business analyst to derive specific functionality that must be allocated to one or another of those component subsystems, as well as demanding an understanding of the interfaces between them.

Business rules include corporate policies, government regulations, industry standards, and computational algorithms. As you'll see in Chapter 9, "Playing by the rules," business rules are not themselves software requirements because they have an existence beyond the boundaries of any specific software application. However, they often dictate that the system must contain functionality to comply with the pertinent rules. Sometimes, as with corporate security policies, business rules are the origin of specific quality attributes that are then implemented in functionality. Therefore, you can trace the genesis of certain functional requirements back to a particular business rule.

In addition to functional requirements, the SRS contains an assortment of nonfunctional requirements. *Quality attributes* are also known as quality factors, quality of service requirements, constraints, and the "-ilities." They describe the product's characteristics in various dimensions that are important either to users or to developers and maintainers, such as performance, safety, availability, and portability. Other classes of nonfunctional requirements describe *external interfaces* between the system and the outside world. These include connections to other software systems, hardware components, and users, as well as communication interfaces. Design and implementation *constraints* impose restrictions on the options available to the developer during construction of the product.

If they're nonfunctional, then what are they?

For many years, the requirements for a software product have been classified broadly as either functional or nonfunctional. The functional requirements are evident: they describe the observable behavior of the system under various conditions. However, many people dislike the term "nonfunctional." That adjective says what the requirements are *not*, but it doesn't say what they *are*. We are sympathetic to the problem, but we lack a perfect solution.

Other-than-functional requirements might specify not *what* the system does, but rather *how well* it does those things. They could describe important characteristics or properties of the system. These include the system's availability, usability, security, performance, and many other characteristics, as addressed in Chapter 14, "Beyond functionality." Some people consider nonfunctional requirements to be synonymous with quality attributes, but that is overly restrictive. For example, design and implementation constraints are also nonfunctional requirements, as are external interface requirements.

Still other nonfunctional requirements address the environment in which the system operates, such as platform, portability, compatibility, and constraints. Many products are also affected by compliance, regulatory, and certification requirements. There could be localization requirements for products that must take into account the cultures, languages, laws, currencies, terminology, spelling, and other characteristics of users. Though such requirements are specified in nonfunctional terms, the business analyst typically will derive numerous bits of functionality to ensure that the system possesses all the desired behaviors and properties.

In this book, we are sticking with the term “nonfunctional requirements,” despite its limitations, for the lack of a suitably inclusive alternative. Rather than worry about precisely what you call these sorts of information, just make sure that they are part of your requirements elicitation and analysis activities. You can deliver a product that has all the desired functionality but that users hate because it doesn’t match their (often unstated) quality expectations.

A *feature* consists of one or more logically related system capabilities that provide value to a user and are described by a set of functional requirements. A customer’s list of desired product features is not equivalent to a description of the user’s task-related needs. Web browser bookmarks, spelling checkers, the ability to define a custom workout program for a piece of exercise equipment, and automatic virus signature updating in an anti-malware product are examples of features. A feature can encompass multiple user requirements, each of which implies that certain functional requirements must be implemented to allow the user to perform the task described by each user requirement. Figure 1-2 illustrates a *feature tree*, an analysis model that shows how a feature can be hierarchically decomposed into a set of smaller features, which relate to specific user requirements and lead to specifying sets of functional requirements (Beatty and Chen 2012).

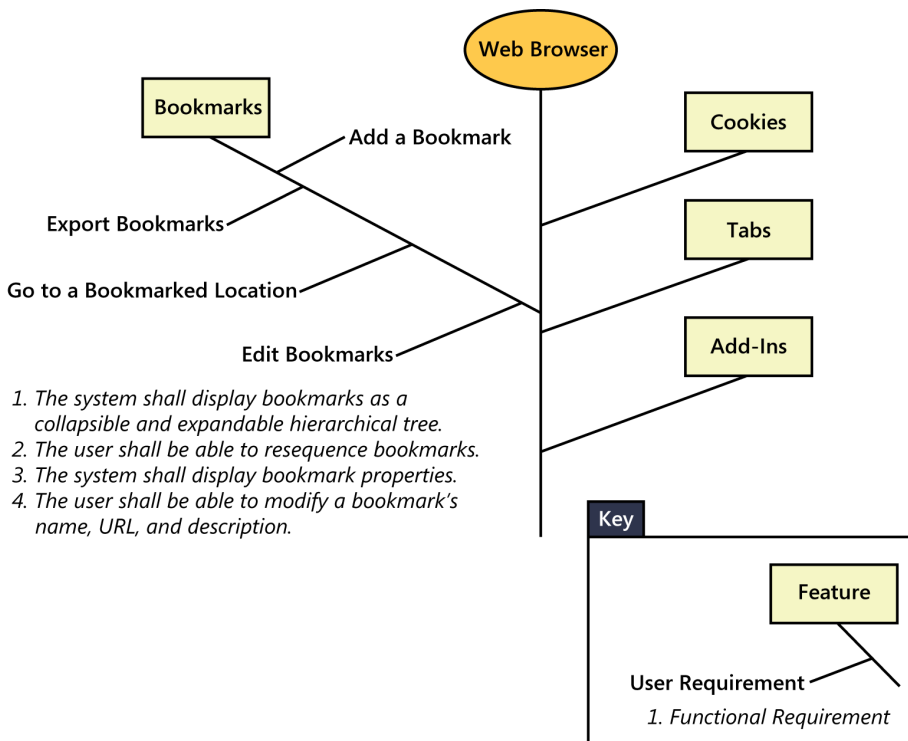


FIGURE 1-2 Relationships among features, user requirements, and functional requirements.

To illustrate some of these various kinds of requirements, consider a project to develop the next version of a text editor program. A business requirement might be “Increase non-US sales by 25 percent within 6 months.” Marketing realizes that the competitive products only have English-language spelling checkers, so they decide that the new version will include a multilanguage spelling checker feature. Corresponding user requirements might include tasks such as “Select language for spelling checker,” “Find spelling errors,” and “Add a word to a dictionary.” The spelling checker has many individual functional requirements, which deal with operations such as highlighting misspelled words, autocorrect, displaying suggested replacements, and globally replacing misspelled words with corrected words. Usability requirements specify how the software is to be localized for use with specific languages and character sets.

Working with the three levels

Figure 1-3 illustrates how various stakeholders might participate in eliciting the three levels of requirements. Different organizations use a variety of names for the roles involved in these activities; think about who performs these activities in your organization. The role names often differ depending on whether the developing organization is an internal corporate entity or a company building software for commercial use.

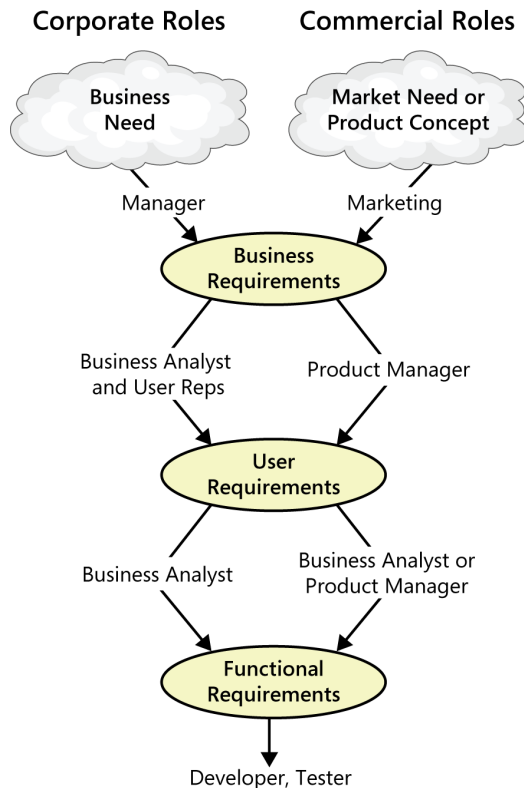


FIGURE 1-3 An example of how different stakeholders participate in requirements development.

Based on an identified business need, a market need, or an exciting new product concept, managers or marketing define the business requirements for software that will help their company operate more efficiently (for information systems) or compete successfully in the marketplace (for commercial products). In the corporate environment, a business analyst then typically works with user representatives to identify user requirements. Companies developing commercial products often identify a product manager to determine what features to include in the new product. Each user requirement and feature must align with accomplishing the business requirements. From the user requirements, the BA or product manager derives the functionality that will let users achieve their goals. Developers use the functional and nonfunctional requirements to design solutions that implement the necessary functionality, within the limits that the constraints impose. Testers determine how to verify whether the requirements were correctly implemented.



It's important to recognize the value of recording vital requirements information in a shareable form, rather than treating it as oral tradition around the project campfire. I was on a project once that had experienced a rotating cast of development teams. The primary customer was sick to tears of having each new team come along and say, "We have to talk about your requirements." His reaction to our request was, "I already gave your predecessors my requirements. Now build me a system!" Unfortunately, no one had ever documented any requirements, so every new team had to start from scratch. To proclaim that you "have the requirements" is delusional if all you really have is a pile of email and voice mail messages, sticky notes, meeting minutes, and vaguely recollected hallway conversations. The BA must practice good judgment to determine just how comprehensive to make the requirements documentation on a given project.

Figure 1-1, shown earlier in this chapter, identified three major requirements deliverables: a vision and scope document, a user requirements document, and a software requirements specification. You do not necessarily need to create three discrete requirements deliverables on each project. It often makes sense to combine some of this information, particularly on small projects. However, recognize that these three deliverables contain different information, developed at different points in the project, possibly by different people, with different purposes and target audiences.

The model in Figure 1-1 showed a simple top-down flow of requirements information. In reality, you should expect cycles and iteration among the business, user, and functional requirements. Whenever someone proposes a new feature, user requirement, or bit of functionality, the analyst must ask, "Is this in scope?" If the answer is "yes," the requirement belongs in the specification. If the answer is "no," it does not, at least not for the forthcoming release or iteration. The third possible answer is "no, but it supports the business objectives, so it ought to be." In that case, whoever controls the project scope—the project sponsor, project manager, or product owner—must decide whether to increase the current project's or iteration's scope to accommodate the new requirement. This is a business decision that has implications for the project's schedule and budget and might demand trade-offs with other capabilities. An effective change process that includes impact analysis ensures that the right people make informed business decisions about which changes to accept and that the associated costs in time, resources, or feature trade-offs are addressed.

Product vs. project requirements

So far we have been discussing requirements that describe properties of a software system to be built. Let's call those *product* requirements. Projects certainly do have other expectations and deliverables that are not a part of the software the team implements, but that are necessary to the successful completion of the project as a whole. These are *project* requirements but not *product* requirements. An SRS houses the product requirements, but it should not include design or implementation details (other than known constraints), project plans, test plans, or similar information. Separate out such items so that requirements development activities can focus on understanding what the team intends to build. Project requirements include:

- Physical resources the development team needs, such as workstations, special hardware devices, testing labs, testing tools and equipment, team rooms, and videoconferencing equipment.
- Staff training needs.
- User documentation, including training materials, tutorials, reference manuals, and release notes.
- Support documentation, such as help desk resources and field maintenance and service information for hardware devices.
- Infrastructure changes needed in the operating environment.
- Requirements and procedures for releasing the product, installing it in the operating environment, configuring it, and testing the installation.
- Requirements and procedures for transitioning from an old system to a new one, such as data migration and conversion requirements, security setup, production cutover, and training to close skills gaps; these are sometimes called *transition requirements* (IIBA 2009).
- Product certification and compliance requirements.
- Revised policies, processes, organizational structures, and similar documents.
- Sourcing, acquisition, and licensing of third-party software and hardware components.
- Beta testing, manufacturing, packaging, marketing, and distribution requirements.
- Customer service-level agreements.
- Requirements for obtaining legal protection (patents, trademarks, or copyrights) for intellectual property related to the software.

This book does not address these sorts of project requirements further. That doesn't mean that they aren't important, just that they are out of scope for our focus on software product requirements development and management. Identifying these project requirements is a shared responsibility of the BA and the project manager. They often come up while eliciting product requirements. Project requirements information is best stored in the project management plan, which should itemize all expected project activities and deliverables.

Particularly for business applications, people sometimes refer to a “solution” as encompassing both the product requirements (which are principally the responsibility of the business analyst) and the project requirements (which are principally the responsibility of the project manager). They might use the term “solution scope” to refer to “everything that has to be done to complete the project successfully.” In this book, though, we are focusing on product requirements, whether your ultimate deliverable is a commercial software product, a hardware device with embedded software, a corporate information system, contracted government software, or anything else.

Requirements development and management

Confusion about requirements terminology extends even to what to call the whole discipline. Some authors call the entire domain *requirements engineering* (our preference). Others refer to it all as *requirements management*. Still others refer to these activities as a subset of the broad domain of business analysis.

We find it useful to split requirements engineering into *requirements development* (addressed in Part II of this book) and *requirements management* (addressed in Part IV), as shown in Figure 1-4. Regardless of what development life cycle your project is following—be it pure waterfall, phased, iterative, incremental, agile, or some hybrid—these are the things you need to do regarding requirements. Depending on the life cycle, you will perform these activities at different times in the project and to varying degrees of depth or detail.

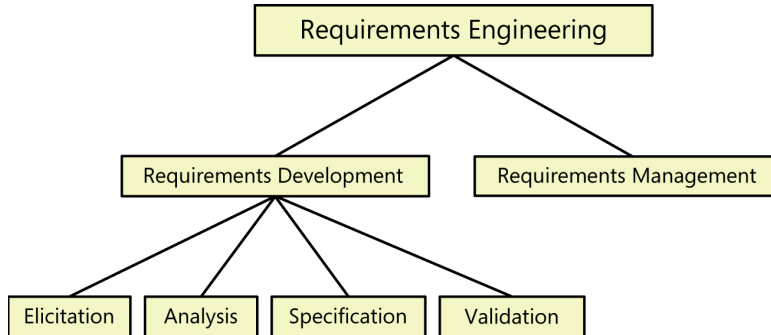


FIGURE 1-4 Subdisciplines of software requirements engineering.

Requirements development

As Figure 1-4 shows, we subdivide requirements development into *elicitation*, *analysis*, *specification*, and *validation* (Abzan et al. 2004). These subdisciplines encompass all the activities involved with exploring, evaluating, documenting, and confirming the requirements for a product. Following are the essential actions in each subdiscipline.

Elicitation

Elicitation encompasses all of the activities involved with discovering requirements, such as interviews, workshops, document analysis, prototyping, and others. The key actions are:

- Identifying the product's expected user classes and other stakeholders.
- Understanding user tasks and goals and the business objectives with which those tasks align.
- Learning about the environment in which the new product will be used.
- Working with individuals who represent each user class to understand their functionality needs and their quality expectations.

Usage-centric or product-centric?

Requirements elicitation typically takes either a usage-centric or a product-centric approach, although other strategies also are possible. The usage-centric strategy emphasizes understanding and exploring user goals to derive the necessary system functionality. The product-centric approach focuses on defining features that you expect will lead to marketplace or business success. A risk with product-centric strategies is that you might implement features that don't get used much, even if they seemed like a good idea at the time. We recommend understanding business objectives and user goals first, then using that insight to determine the appropriate product features and characteristics.

Analysis

Analyzing requirements involves reaching a richer and more precise understanding of each requirement and representing sets of requirements in multiple ways. Following are the principal activities:

- Analyzing the information received from users to distinguish their task goals from functional requirements, quality expectations, business rules, suggested solutions, and other information
- Decomposing high-level requirements into an appropriate level of detail
- Deriving functional requirements from other requirements information
- Understanding the relative importance of quality attributes
- Allocating requirements to software components defined in the system architecture
- Negotiating implementation priorities
- Identifying gaps in requirements or unnecessary requirements as they relate to the defined scope

Specification

Requirements specification involves representing and storing the collected requirements knowledge in a persistent and well-organized fashion. The principal activity is:

- Translating the collected user needs into written requirements and diagrams suitable for comprehension, review, and use by their intended audiences.

Validation

Requirements validation confirms that you have the correct set of requirements information that will enable developers to build a solution that satisfies the business objectives. The central activities are:

- Reviewing the documented requirements to correct any problems before the development group accepts them.
- Developing acceptance tests and criteria to confirm that a product based on the requirements would meet customer needs and achieve the business objectives.

Iteration is a key to requirements development success. Plan for multiple cycles of exploring requirements, progressively refining high-level requirements into more precision and detail, and confirming correctness with users. This takes time and it can be frustrating. Nonetheless, it's an intrinsic aspect of dealing with the fuzzy uncertainty of defining a new software system.



Important You're never going to get perfect requirements. From a practical point of view, the goal of requirements development is to accumulate a shared understanding of requirements that is *good enough* to allow construction of the next portion of the product—be that 1 percent or 100 percent of the entire product—to proceed at an acceptable level of risk. The major risk is that of having to do excessive unplanned rework because the team didn't sufficiently understand the requirements for the next chunk of work before starting design and construction.

Requirements management

Requirements management activities include the following:

- Defining the requirements baseline, a snapshot in time that represents an agreed-upon, reviewed, and approved set of functional and nonfunctional requirements, often for a specific product release or development iteration
- Evaluating the impact of proposed requirements changes and incorporating approved changes into the project in a controlled way
- Keeping project plans current with the requirements as they evolve
- Negotiating new commitments based on the estimated impact of requirements changes

- Defining the relationships and dependencies that exist between requirements
- Tracing individual requirements to their corresponding designs, source code, and tests
- Tracking requirements status and change activity throughout the project

The object of requirements management is not to stifle change or to make it difficult. It is to anticipate and accommodate the very real changes that you can always expect so as to minimize their disruptive impact on the project.

Figure 1-5 provides another view of the boundary between requirements development and requirements management. This book describes dozens of specific practices for performing requirements elicitation, analysis, specification, validation, and management.

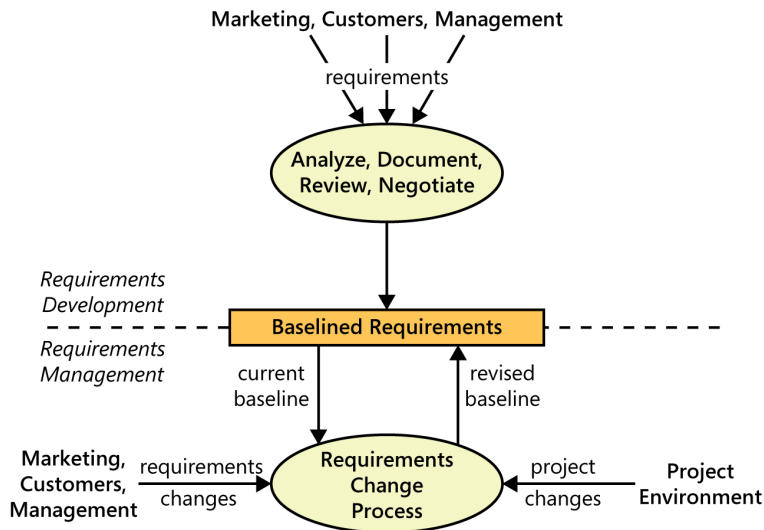


FIGURE 1-5 The boundary between requirements development and requirements management.

Every project has requirements

Frederick Brooks eloquently stated the critical role of requirements to a software project in his classic 1987 essay, "No Silver Bullet: Essence and Accidents of Software Engineering":

The hardest single part of building a software system is deciding precisely what to build. No other part of the conceptual work is as difficult as establishing the detailed technical requirements, including all the interfaces to people, to machines, and to other software systems. No other part of the work so cripples the resulting system if done wrong. No other part is more difficult to rectify later.

Every software-containing system has stakeholders who rely on it. The time spent understanding their needs is a high-leverage investment in project success. If a project team does not have written representations of requirements that the stakeholders agree to, how can developers be sure to satisfy those stakeholders?

Often, it's impossible—or unnecessary—to fully specify the functional requirements before commencing design and implementation. In those cases, you can take an iterative or incremental approach, implementing one portion of the requirements at a time and obtaining customer feedback before moving on to the next cycle. This is the essence of agile development, learning just enough about requirements to do thoughtful prioritization and release planning so the team can begin delivering valuable software as quickly as possible. This isn't an excuse to write code before contemplating requirements for that next increment, though. Iterating on code is more expensive than iterating on concepts.

People sometimes balk at spending the time that it takes to write software requirements. But writing the requirements isn't the hard part. The hard part is *determining* the requirements. Writing requirements is a matter of clarifying, elaborating, and recording what you've learned. A solid understanding of a product's requirements ensures that your team works on the right problem and devises the best solution to that problem. Without knowing the requirements, you can't tell when the project is done, determine whether it has met its goals, or make trade-off decisions when scope adjustments are necessary. Instead of balking at spending time on requirements, people should instead balk at the money wasted when the project doesn't pay enough attention to requirements.

When bad requirements happen to good people

The major consequence of requirements problems is rework—doing again something that you thought was already done—late in development or after release. Rework often consumes 30 to 50 percent of your total development cost (Shull, et al. 2002; GAO 2004), and requirements errors can account for 70 to 85 percent of the rework cost (Leffingwell 1997). Some rework does add value and improves the product, but excessive rework is wasteful and frustrating. Imagine how different your life would be if you could cut the rework effort in half! Your team members could build better products faster and perhaps even go home on time. Creating better requirements is an investment, not just a cost.

It can cost far more to correct a defect that's found late in the project than to fix it shortly after its creation. Suppose it costs \$1 (on a relative scale) to find and fix a requirement defect while you're still working on the requirements. If you discover that error during design instead, you have to pay the \$1 to fix the requirement error, plus another \$2 or \$3 to redo the design that was based on the incorrect requirement. Suppose, though, that no one finds the error until a user calls with a problem. Depending on the type of system, the cost to correct a requirement defect found in operation can be \$100 or more on this relative scale (Boehm 1981; Grady 1999; Haskins 2004). One of my consulting clients determined that they spent an average of \$200 of labor effort to find and fix a defect in their information systems using the quality technique of software inspection, a type of peer review



(Wiegers 2002). In contrast, they spent an average of \$4,200 to fix a single defect reported by the user, an amplification factor of 21. Preventing requirements errors and catching them early clearly has a huge leveraging effect on reducing rework.

Shortcomings in requirements practices pose many risks to project success, where *success* means delivering a product that satisfies the user's functional and quality expectations at the agreed-upon cost and schedule. Chapter 32, "Software requirements and risk management," describes how to manage such risks to prevent them from derailing your project. Some of the most common requirements risks are described in the following sections.

Insufficient user involvement

Customers often don't understand why it is so essential to work hard on eliciting requirements and assuring their quality. Developers might not emphasize user involvement, perhaps because they think they already understand what the users need. In some cases it's difficult to gain access to people who will actually use the product, and user surrogates don't always understand what users really need. Insufficient user involvement leads to late-breaking requirements that generate rework and delay completion.

Another risk of insufficient user involvement, particularly when reviewing and validating the requirements, is that the business analyst might not understand and properly record the true business or customer needs. Sometimes a BA goes down the path of specifying what appears to be the "perfect" requirements, and developers implement them, but then no one uses the solution because the business problem was misunderstood. Ongoing conversations with users can help mitigate this risk, but if users don't review the requirements carefully enough, you can still have problems.

Inaccurate planning

"Here's my idea for a new product; when will you be done?" No one should answer this question until more is known about the problem being discussed. Vague, poorly understood requirements lead to overly optimistic estimates, which come back to haunt you when the inevitable overruns occur. An estimator's quick guess sounds a lot like a commitment to the listener. The top contributors to poor software cost estimation are frequent requirements changes, missing requirements, insufficient communication with users, poor specification of requirements, and insufficient requirements analysis (Davis 1995). Estimating project effort and duration based on requirements means that you need to know something about the size of your requirements and the development team's productivity. See Chapter 5 of *More about Software Requirements* (Wiegers 2006) for more about estimation based on requirements.

Creeping user requirements

As requirements evolve during development, projects often exceed their planned schedules and budgets (which are nearly always too optimistic anyway). To manage scope creep, begin with a clear statement of the project's business objectives, strategic vision, scope, limitations, and success criteria. Evaluate all proposed new features or requirements changes against this reference. Requirements *will*

change and grow. The project manager should build contingency buffers into schedules so the first new requirement that comes along doesn't derail the schedule (Wiegers 2007). Agile projects take the approach of adjusting the scope for a certain iteration to fit into a defined budget and duration for the iteration. As new requirements come along, they are placed into the backlog of pending work and allocated to future iterations based on priority. Change might be critical to success, but change always has a price.

Ambiguous requirements

One symptom of ambiguity in requirements is that a reader can interpret a requirement statement in several ways (Lawrence 1996). Another sign is that multiple readers of a requirement arrive at different understandings of what it means. Chapter 11, "Writing excellent requirements," lists many words and phrases that contribute to ambiguity by placing the burden of interpretation on the reader.

Ambiguity leads to different expectations on the part of various stakeholders. Some of them are then surprised at whatever is delivered. Ambiguous requirements cause wasted time when developers implement a solution for the wrong problem. Testers who expect the product to behave differently from what the developers built waste time resolving the differences.

One way to ferret out ambiguity is to have people who represent different perspectives inspect the requirements (Wiegers 2002). As described in Chapter 17, "Validating the requirements," informal peer reviews in which reviewers simply read the requirements on their own often don't reveal ambiguities. If different reviewers interpret a requirement in different ways but it makes sense to each of them, they won't find the ambiguity. Collaborative elicitation and validation encourages stakeholders to discuss and clarify requirements as a group in a workshop setting. Writing tests against the requirements and building prototypes are other ways to discover ambiguities.

Gold plating

Gold plating takes place when a developer adds functionality that wasn't in the requirements specification (or was deemed out of scope) but which the developer believes "the users are just going to love." If users don't care about this functionality, the time spent implementing it is wasted. Rather than simply inserting new features, developers and BAs should present stakeholders with creative ideas for their consideration. Developers should strive for leanness and simplicity, not going beyond what stakeholders request without their approval.

Customers sometimes request certain features or elaborate user interfaces that look attractive but add little value to the product. Everything you build costs time and money, so you need to maximize the delivered value. To reduce the threat of gold plating, trace each bit of functionality back to its origin and its business justification so everyone knows why it's included. Make sure that what you are specifying and developing lies within the project's scope.

Overlooked stakeholders

Most products have several groups of users who might use different subsets of features, have different frequencies of use, or have varying levels of experience. If you don't identify the important user classes for your product early on, some user needs won't be met. After identifying all user classes, make sure that each has a voice, as discussed in Chapter 6, "Finding the voice of the user." Besides obvious users, think about maintenance and field support staff who have their own requirements, both functional and nonfunctional. People who have to convert data from a legacy system will have transition requirements that don't affect the ultimate product software but that certainly influence solution success. You might have stakeholders who don't even know the project exists, such as government agencies that mandate standards that affect your system, yet you need to know about them and their influence on the project.

Benefits from a high-quality requirements process

Some people mistakenly believe that time spent discussing requirements simply delays delivery by the same duration. This assumes that there's no return on investment from requirements activities. In actuality, investing in good requirements will virtually always return more than it costs.

Sound requirements processes emphasize a collaborative approach to product development that involves stakeholders in a partnership throughout the project. Eliciting requirements lets the development team better understand its user community or market, a critical success factor. Emphasizing user tasks instead of superficially attractive features helps the team avoid writing code that no one will ever execute. Customer involvement reduces the expectation gap between what the customer really needs and what the developer delivers. You're going to get the customer input eventually; it's far cheaper to reach this understanding before you build the product than after delivery. Chapter 2 addresses the nature of the customer-development partnership.

Explicitly allocating system requirements to various software, hardware, and human subsystems emphasizes a systems approach to product engineering. An effective change control process will minimize the adverse impact of requirements changes. Documented and clear requirements greatly facilitate system testing. All of these increase your chances of delivering high-quality products that satisfy all stakeholders.

No one can promise a specific return on investment from using sound requirements practices. You can go through an analytical thought process to imagine how better requirements could help your teams, though (Wiegiers 2006). The cost of better requirements includes developing new procedures and document templates, training the team, and buying tools. Your greatest investment is the time your project teams actually spend on requirements engineering tasks. The potential payoff includes:

- Fewer defects in requirements and in the delivered product.
- Reduced development rework.
- Faster development and delivery.

- Fewer unnecessary and unused features.
- Lower enhancement costs.
- Fewer miscommunications.
- Reduced scope creep.
- Reduced project chaos.
- Higher customer and team member satisfaction.
- Products that do what they're supposed to do.

Even if you can't quantify all of these benefits, they are real.



Next steps

- Write down requirements-related problems that you have encountered on your current or previous project. Identify each as a requirements development or requirements management problem. Describe the root cause of each problem and its impact on the project.
- Facilitate a discussion with your team members and other stakeholders regarding requirements-related problems from your current or previous projects, their impacts, and their root causes. Pool your ideas about changes in your current requirements practices that could address these problems. The troubleshooting guide in Appendix B might be helpful.
- Map the requirements terminology and deliverables used in your organization to that shown in this chapter to see if you're covering all the categories recommended here.
- Perform a simple assessment on just a few pages of one of your requirements documents to see where your team might have some clear improvement areas. It might be most useful to have an objective outsider perform this assessment.
- Arrange a training class on software requirements for your entire project team. Invite key customers, marketing staff, managers, developers, testers, and other stakeholders to participate. Training gives project participants a common vocabulary. It provides a shared appreciation of effective techniques and behaviors so that all team members can collaborate more effectively on their mutual challenges.

